

Spring 2026: ME759 Final Project Proposal

Quick intro:

- Name: Md Zubayer Yeaser
- Email: yeaser@wisc.edu
- Home department: Civil and Environmental Engineering
- Status: PhD Student.
- Name of your teammate (if applicable): N/A

Project Title: GPU-Accelerated Triangular Shell Finite Element Solver via Custom CUDA Kernels in Julia

Link to git repo for project: <https://github.com/zyeaser/repo759>

Problem statement: Finite element analysis of thin shell structures using triangular shell elements is computationally expensive, particularly for large meshes encountered in structural mechanics and engineering simulations. The existing TriShellFiniteElement.jl implementation (<https://github.com/runtosolve/TriShellFiniteElement.jl/>), built on top of the Ferrite.jl FEM framework in Julia, runs entirely on the CPU. This project aims to GPU-accelerate the core bottlenecks of this pipeline, namely element-level stiffness matrix computation, global assembly, and linear system solve, using custom CUDA kernels written with CUDA.jl. The goal is to achieve significant speedup over the serial CPU baseline without resorting to high-level GPU wrapper libraries, demonstrating the performance benefits of hand-written, architecture-aware GPU code.

Motivation/Rationale: A direct motivation for this project is the recently published work by Moen, Ádány, and Shabhari (SSRC Annual Stability Conference, April 2026), which introduces TriShellFiniteElement.jl, an open-source triangular Mindlin shell element implemented in Julia using Ferrite.jl. That paper demonstrates the formulation matches commercial ABAQUS S4R/S3R solutions for membrane, bending, and buckling behavior of thin-walled structures, and explicitly identifies performance at scale as the next frontier, calling out the need to showcase how much faster these open-source packages run on large models versus commercial FEM programs. This project directly answers that challenge by writing custom CUDA.jl kernels for stiffness assembly and the linear solver, rather than relying on high-level GPU wrappers.

Explain how you contemplate going about it: GPU parallel computing via CUDA.jl in Julia, with all kernels written from scratch. The three core areas are: (1) element stiffness matrix computation, where each GPU thread handles one shell element independently; (2) global stiffness assembly into a sparse matrix format, with parallel kernels managing contributions from all elements; and (3) a PCG linear solver ($K \cdot u = f$) built from individual GPU kernels, including sparse matrix-vector product, dot product, and vector operations. Ferrite.jl handles mesh setup and DOF management on the CPU. The specific kernel designs and optimizations will evolve as the implementation progresses.

ME759 aspects the proposed work draws on: GPU thread/block/grid hierarchy and occupancy tuning; shared memory usage for element-local data reuse; atomic operations for race-free parallel assembly; memory coalescing in sparse matrix kernels; performance profiling and roofline analysis; custom iterative linear solvers on GPU.

Deliverables: By 05/05/2026 at 9:45 AM: (1) Julia source code with custom CUDA kernels for stiffness assembly and PCG solver; (2) Benchmark results reproducing the same test cases from Moen et al. (SSRC 2026) — out-of-plane bending (midspan and uniform pressure), in-plane bending (midspan and uniform pressure), and column buckling — comparing GPU, CPU (TriShellFiniteElement.jl default), and the published Abaqus reference values, across the same mesh sizes used in the paper; (3) Speedup plots showing GPU vs. CPU runtime across mesh sizes for each test case; (4) Technical report documenting the implementation and performance results.

How you will demonstrate what you accomplished: The benchmark test cases from Moen et al. (SSRC 2026), out-of-plane bending, in-plane bending, and column buckling, will be re-run using the GPU implementation and compared against the published CPU results and Abaqus reference values, across the same mesh sizes used in the paper. This directly demonstrates both correctness and speedup in a well-defined, reproducible setting. Beyond ME759, this GPU solver is a building block toward a broader goal: establishing a fully open-source, high-performance FEM platform for structural engineering and stability research, where the community can freely explore and extend shell element formulations without the cost or opacity of commercial software. The linear static GPU implementation completed here is the first concrete step in that direction, with nonlinear solvers and plasticity as natural extensions for future dissertation work.

Other remarks: I have access to the Euler's GPU cluster for running experiments at scale. The TriShellFiniteElement.jl CPU codebase already exists and is functional, so the project is focused entirely on the GPU implementation rather than developing the FEM formulation from scratch. All CUDA kernels will be original code written in Julia using CUDA.jl. No high-level GPU FEM libraries will be used.